

1. Abertura da Aula

Olá, equipe! Bem-vindos à nossa 11ª aula do curso de Desenvolvimento de Sistemas. Hoje vamos conectar o design visual com a engenharia de front-end, focando em UI Design Aplicado e na construção de formulários interativos. No contexto da nossa Situação de Aprendizagem, vocês estão desenvolvendo uma aplicação web inovadora para apresentar a investidores anjo. Pensem no seguinte: a porta de entrada de qualquer sistema, a forma como o usuário entrega dados para a sua aplicação, é o formulário. Seja na tela de Login, no Cadastro de novos usuários ou no painel de CRUD (Create, Read, Update, Delete) de produtos, um formulário mal construído, confuso ou inacessível destrói a experiência do usuário e afasta investimentos. Hoje, nossa missão é transformar os protótipos visuais de vocês (feitos no Figma) em código HTML5 semântico, aplicando validações nativas, garantindo acessibilidade e verificando o contraste visual para entregar um produto de nível profissional ao mercado.

2. Conceitos Fundamentais

Conceito 1: Estrutura Semântica e Tipos de Input no HTML5

Explicação Teórica:

Um formulário web é delimitado pela tag `<form>`, que atua como um contêiner para os dados que serão enviados ao servidor (nosso futuro back-end em PHP). Dentro desse contêiner, utilizamos a tag `<input>` para coletar informações. O grande poder do HTML5 no mercado atual é o atributo `type`. Antigamente, tudo era apenas texto (`type="text"`). Hoje, ao especificarmos `type="email"`, `type="password"`, `type="date"` ou `type="number"`, o navegador automaticamente entende o formato do dado esperado. Mais importante ainda: em dispositivos móveis (como os celulares dos nossos investidores), definir o tipo correto faz com que o teclado virtual do celular mude dinamicamente (por exemplo, exibindo o botão "@" e ".com" para e-mails, ou apenas o teclado numérico para números), o que é um requisito fundamental de usabilidade (CT01).

Demonstração Prática:

Vejamos a criação de uma estrutura básica de captura de dados de acesso.

HTML

```
<form action="/login.php" method="POST">
  <input type="email" name="emailUsuario" id="emailUsuario">

  <input type="password" name="senhaUsuario" id="senhaUsuario">

  <button type="submit">Entrar</button>
</form>
```

Explicação do código: - A tag `<form>` define o início do formulário. Os atributos `action` e `method` indicam para onde e como os dados irão (veremos no back-end).

- O `<input type="email">` obriga o usuário a digitar um formato válido de e-mail (com @ e domínio). O atributo `name` é como o PHP reconhecerá essa variável depois, e o `id` é exclusivo para o front-end.
- O `<input type="password">` oculta os caracteres digitados por questões de segurança.
- O `<button type="submit">` é o gatilho que empacota os dados e os envia.

Exercício Guiado:

Abram o editor de código. Crie um arquivo `pratica.html`. Monte um `<form>` que contenha um input para o nome completo do usuário, um input para a data de nascimento e um botão de envio. Utilize os tipos de input corretos para cada dado.

Exercício de Fixação:

Expanda o formulário anterior: adicione um campo para "Idade" (usando o tipo adequado para números) e um campo para "Telefone" (pesquise e utilize o tipo adequado para telefones no HTML5).

Conceito 2: Atributos de Usabilidade e Acessibilidade (Label, Placeholder e Required)

Explicação Teórica:

Acessibilidade digital significa garantir que pessoas com deficiência (como pessoas com deficiência visual que usam leitores de tela) consigam usar o seu sistema. No mercado corporativo, isso não é apenas empatia, é requisito legal. Para isso, todo `<input>` deve ter um rótulo associado usando a tag `<label>`. Nós conectamos o `label` ao `input` fazendo com que o atributo `for` do `label` seja exatamente igual ao atributo `id` do `input`. Além disso, utilizamos o `placeholder` para dar uma dica visual leve dentro da caixa de texto, e o atributo booleano `required` para avisar ao navegador que o formulário não pode ser enviado se aquele campo estiver vazio.

Demonstração Prática:

Vamos aplicar usabilidade e acessibilidade a um campo de cadastro.

HTML

```
<div class="form-group">
  <label for="nomeCliente">Nome Completo</label>

  <input type="text" id="nomeCliente" name="nomeCliente"
    placeholder="Ex: Maria da Silva" required>
</div>
```

Explicação do código: - A tag `<label for="nomeCliente">` cria um texto clicável. Se o usuário clicar na palavra "Nome Completo", o cursor pulará para dentro do input. Leitores de tela lerão esse rótulo em voz alta.

- No `<input>`, o `id="nomeCliente"` estabelece o link com o `label`.
- O `placeholder="Ex: Maria da Silva"` mostra uma sugestão de preenchimento que some assim que o usuário começa a digitar.
- A palavra `required` bloqueia o envio do form se o usuário tentar pular essa etapa.

Exercício Guiado:

Volte ao formulário que você criou no exercício anterior. Adicione tags `<label>` para todos os seus inputs, garantindo que o atributo `for` esteja corretamente ligado ao `id`.

Exercício de Fixação:

Adicione dicas de preenchimento (placeholders) em todos os campos de texto. Em seguida, torne o campo de "Nome Completo" e "Data de nascimento" obrigatórios adicionando o atributo `required`. Tente submeter o formulário vazio no navegador para ver o que acontece.

Conceito 3: Validação HTML5 e Verificação de Contraste WCAG

Explicação Teórica:

A validação no front-end evita que dados inúteis ou perigosos viajem até o servidor, economizando banda e processamento. O HTML5 faz a validação primária (como verificar se um e-mail é válido). Outro ponto crítico de UI Design é a verificação de contraste de cores, regida pelas diretrizes WCAG (*Web Content Accessibility Guidelines*). Um texto cinza claro sobre um fundo branco pode parecer bonito no seu protótipo do Figma, mas será ilegível para um usuário com baixa visão, ou para o investidor lendo a tela do celular sob a luz do sol. O mercado exige ferramentas online para medir a taxa de contraste (o mínimo exigido por lei para textos normais é uma proporção de 4.5:1).

Demonstração Prática:

Vamos combinar uma validação mais restrita com a estilização focada em legibilidade.

HTML

```
<label for="cpf">CPF (apenas números)</label>
<input type="text" id="cpf" name="cpf" pattern="\d{11}"
      title="Digite exatamente 11 números" required
      style="background-color: #ffffff; color: #333333; border: 2px solid #0056b3;">
```

Explicação do código: - O atributo `pattern="\d{11}"` usa uma Expressão Regular para obrigar que o campo receba exatamente 11 dígitos numéricos.

- O `title` fornece a mensagem de erro personalizada se o usuário errar o padrão.
- No `style` (que na vida real vai para o arquivo CSS), as cores `#ffffff` (fundo branco) e `#333333` (texto cinza muito escuro) garantem uma relação de contraste altíssima (21:1), passando nos testes WCAG nível AAA.

Exercício Guiado:

Crie um campo `<input type="password">` para a criação de uma senha. Adicione o atributo `minlength="8"` para obrigar que a senha tenha no mínimo 8 caracteres.

Exercício de Fixação:

Abra o site [WebAIM Contrast Checker](#). Pegue as cores principal (fundo) e secundária (texto dos botões) que sua equipe definiu no protótipo do Figma. Insira os códigos Hexadecimais na ferramenta e verifique se a taxa é superior a 4.5:1. Se falhar, ajuste a cor até passar e anote o novo código.

3. Demonstração Aplicada

Vamos consolidar tudo criando a estrutura principal de um Formulário CRUD de Produto, integrando com as classes do Bootstrap da aula passada, acessibilidade e validação HTML5.

HTML

```

<form class="container mt-4 border p-4 bg-light rounded shadow-sm" action="#" method="POST">
  <h3 class="mb-4 text-primary">Cadastro de Novo Produto</h3>

  <div class="row">
    <div class="col-md-6 mb-3">
      <label for="produtoNome" class="form-label fw-bold">Nome do Produto *</label>
      <input type="text" class="form-control" id="produtoNome" name="produtoNome" placeholder="Ex:
Mouse sem fio" required>
    </div>

    <div class="col-md-6 mb-3">
      <label for="produtoPreco" class="form-label fw-bold">Preço de Venda (R$) *</label>
      <input type="number" class="form-control" id="produtoPreco" name="produtoPreco" step="0.01"
min="0" placeholder="0.00" required>
    </div>
  </div>

  <div class="mb-3">
    <label for="produtoDescricao" class="form-label fw-bold">Descrição Técnica</label>
    <textarea class="form-control" id="produtoDescricao" name="produtoDescricao" rows="4"
placeholder="Detalhes completos do item..."></textarea>
  </div>

  <button type="submit" class="btn btn-primary w-100 fs-5">Salvar Produto</button>
</form>

```

Resultado Esperado: O navegador renderizará um cartão bem espaçado. Se o usuário clicar em "Salvar Produto" com os campos em branco, o próprio Chrome/Edge emitirá um balão vermelho apontando para o "Nome do Produto" dizendo "Preencha este campo". O código é limpo, altamente acessível para leitores de tela devido às labels, e possui os tipos corretos para segurança básica no front-end.

4. Casos Reais do Mercado

- **Checkout de E-commerce (Carrinho Abandonado):** Uma das maiores perdas de receita em lojas virtuais é o "abandono de carrinho". O usuário quer comprar, mas o formulário pede dados demais, não tem placeholder claro, o celular não abre o teclado numérico na hora de digitar o cartão de crédito e a pessoa desiste. Empresas como a Amazon otimizam essas tags `<input>` ao extremo para reduzir a fricção e aumentar o lucro.
- **Sistemas Governamentais e Bancários (Acessibilidade):** Bancos e sites de governos são obrigados por lei a seguir as diretrizes WCAG. Se um idoso ou deficiente visual não conseguir ler a tela de login devido ao baixo contraste, ou se o leitor de tela não encontrar as tags `<label>`, a instituição financeira pode sofrer processos milionários. O front-end carrega a responsabilidade legal de garantir a inclusão digital.

5. FAQ -- Dúvidas Reais de Alunos

1. Qual a diferença entre id e name em um `<input>`?

- O `id` é usado no Front-end (para conectar a `<label>` e para estilizar no CSS ou manipular no JavaScript). Ele deve ser único na página. O `name` é usado no Back-end; é o nome da "variável" que o PHP vai receber quando o formulário for enviado.

2. **Por que devo usar `<label>` se eu posso apenas colocar um parágrafo `<p>` acima do input?**
 - Por usabilidade e acessibilidade. O parágrafo não é clicável. A `<label>` aumenta a área de clique (ótimo para celulares) e avisa aos leitores de tela para deficientes visuais a qual campo aquele texto pertence.
3. **Se o HTML5 já tem validação (como o `required`), eu não preciso validar no back-end (PHP), certo?**
 - Errado! A validação Front-end é apenas para conforto do usuário. Um usuário mal-intencionado pode desligar o HTML, remover o `required` e enviar dados vazios. A verdadeira segurança dos dados sempre acontece no Back-end (PHP). O front-end é a primeira barreira.
4. **O que acontece se a cor que eu escolhi no Figma reprovar no teste WCAG?**
 - Você, como desenvolvedor front-end ou UI Designer, deve ajustar a cor. Geralmente, basta escurecer um pouco o tom do texto ou clarear um pouco o fundo na paleta de cores até que a ferramenta indique o "PASS" (taxa superior a 4.5:1).
5. **Posso mudar a mensagem padrão do navegador ("Preencha este campo")?**
 - Sim, mas para fazer isso de forma customizada e bonita, geralmente desativamos a validação nativa com o atributo `novalidate` na tag `<form>` e usamos JavaScript para exibir mensagens personalizadas na tela. Veremos isso nas próximas aulas!

6. Desafio Prático

Contexto:

Agora, vocês devem transformar as telas estáticas que desenharam ou planejaram em componentes vivos. A missão da equipe é construir a interface de interação de dados do nosso sistema da Situação de Aprendizagem.

Requisitos:

Vocês devem criar **três arquivos HTML independentes**:

1. `login.html`: Um formulário centralizado contendo E-mail, Senha e botão de Acesso.
2. `register.html`: Formulário de criação de conta contendo Nome, E-mail, Senha, Confirmação de Senha e um Checkbox para "Aceite os Termos".
3. `form-crud.html`: Um formulário rico para cadastro de produtos ou serviços (dependendo do tema da equipe), contendo no mínimo: inputs de texto, números, um select (lista suspensa) e um campo de data.

Diretrizes:

- Aplicar o CSS/Bootstrap da aula passada para deixar o layout profissional.
- Todos os inputs devem ter `id`, `name`, `type` correto, `placeholder` e `label` associada.
- Campos cruciais devem usar o atributo `required`.
- O contraste das cores de fundo e texto dos botões deve ser obrigatoriamente aprovado na ferramenta online do WCAG.

Resultado Esperado:

Os três arquivos HTML funcionais no navegador. Ao tentar submeter o formulário vazio, o navegador deve barrar a ação. O código-fonte deve estar limpo e semântico.

Sugestões de Solução:

Dividam as tarefas na equipe (CS1 - Trabalho em equipe). Enquanto um desenvolvedor monta a tela de Login, outro foca no Cadastro e outro no CRUD. Após finalizarem, revisem o código uns dos outros verificando se esqueceram das tags `<label>`.

7. Conexão com a Situação de Aprendizagem

Este é um momento chave do nosso projeto final. A entrega exige o desenvolvimento e implantação de um sistema web funcional acompanhado de documentação (CT01, CT02). Os formulários que vocês acabaram de criar (Login, Register e CRUD) são exatamente a ponte de comunicação que ligará a interface gráfica com o banco de dados que estruturaremos no futuro. Garantir que a validação HTML5 funcione e que o contraste esteja adequado atende diretamente aos requisitos não funcionais de Usabilidade e Qualidade da nossa rubrica de avaliação. Ao exibir isso aos investidores, a equipe demonstra maturidade e compromisso com o design centrado no usuário.

8. Preparação para a Próxima Aula

Hoje nós deixamos nossos formulários inteligentes até onde o HTML permite. Mas e se quisermos fazer algo mais complexo? Por exemplo: verificar se o campo "Senha" e "Confirmar Senha" são idênticos *antes* do usuário clicar no botão de enviar? Ou calcular automaticamente o valor de desconto de um produto enquanto o usuário digita no formulário CRUD? O HTML puro não consegue fazer matemática nem tomar decisões lógicas complexas. É por isso que, na nossa próxima aula, abriremos a **Unidade 3 - Interatividade e Lógica Client-Side**. Vamos ser introduzidos ao maravilhoso mundo do **JavaScript**. Preparem-se para dar inteligência real e dinamismo total aos elementos de tela que construímos até aqui!